NAME: YELE CHARAN

SRN:PES1UG25CS856

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337
639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  .gitignore
  untracked:  README.md
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  tree.h
  untracked:  index.h
  untracked:  Makefile
  untracked:  index.c
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17766800
82 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 17766800
86 6 file2.txt
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./pes commit -m "Initial commit"
Committed: 0aa12f1d1d70... Initial commit
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./pes commit -m "Add world"
Committed: 7f2893a413bf... Add world
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ echo "Goodbye" > bye.txt
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./pes add bye.txt
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./pes commit -m "Add farewell"
Committed: 5cf41b418c39... Add farewell
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ ./pes log
commit 5cf41b418c39b61fcb54f609c881060d7abbbb33963c09c11f056a7766cb95f6
Author: CHARAN <PES1UG25CS856>
Date:   1776680569

    Add farewell

commit 7f2893a413bf17ef38db8da4c9a9d64ee8efbb93130eee49eb3975cb676a1e04
Author: CHARAN <PES1UG25CS856>
Date:   1776680523

    Add world

commit 0aa12f1d1d70c44ffcdee0b05f423b0ee3cba8610a49cf919ab0251135f5c44a
Author: CHARAN <PES1UG25CS856>
Date:   1776680388

    Initial committial commit
```

```
e-problem-main$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0a/a12f1d1d70c44ffcdee0b05f423b0ee3cba8610a49cf919ab0251135f5c44a
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/5c/f41b418c39b61fcb54f609c881060d7abbbb33963c09c11f056a7766cb95f6
.pes/objects/7f/2893a413bf17ef38db8da4c9a9d64ee8efbb93130eee49eb3975cb676a1e04
.pes/objects/b4/e76aee38854839557e5a3132e346adbf8288ec2ba878f2cd915d5c2326af00
.pes/objects/c7/187e8fdb691b3a692e5f3f0bbcb6359e5046285225f18f9773d4fe54268c55
.pes/objects/e4/329ff9662ebec3c7589983c9c43219d408b4de5c3cb6305ddca11ffd1b102d
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f6/8aeaab18272fa2c862dc8721e1ddfbaba520a9b87953adeeb24a7d9412407f
.pes/refs/heads/main
```

```
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ cat .pes/HEAD
ref: refs/heads/main
yele-charan@yele-charan-VirtualBox:~/Downloads/PES1UG25CS856-pes-vcs/os-u4-orang
e-problem-main$ cat .pes/refs/heads/main
5cf41b418c39b61fcb54f609c881060d7abbbb33963c09c11f056a7766cb95f6
```

Q5.1 — How to implement pes checkout <branch>?

To checkout a branch, three things need to change in .pes/: HEAD must be updated to point to the new branch (e.g. ref: refs/heads/newbranch), and the working directory files must be updated to match the target branch's tree. You'd read the target branch's commit, get its

tree, then for every file in that tree write it back to disk. What makes it complex is handling files that exist in the current branch but not the target — those need to be deleted. Also nested subdirectories need recursive handling.

Q5.2 — Detecting dirty working directory conflicts?

Load the current index and compare each entry's stored mtime and size against the actual file on disk using stat(). If any file that differs between the two branches has been modified (mtime or size mismatch between index and working directory), refuse the checkout and tell the user. This is exactly the fast-path diff that index_status already does — no re-hashing needed.

Q5.3 — Detached HEAD?

In detached HEAD state, HEAD contains a raw commit hash instead of ref: refs/heads/branchname. Any new commits you make update HEAD directly but no branch pointer moves. If you then switch branches, those commits become unreachable — no branch points to them. To recover them, you'd need to know the commit hash (from terminal history or reflog) and run pes checkout -b newbranch <hash> to create a branch pointing to them before they get garbage collected.

Q6.1 — Garbage collection algorithm?

Start from all branch refs in .pes/refs/heads/. For each, walk the commit chain following parent pointers, collecting every commit hash. For each commit, read its tree and recursively collect all tree and blob hashes. Store all reachable hashes in a hash set. Then scan every file in .pes/objects/ and delete any whose hash is not in the set. For 100,000 commits with 50 branches you'd need to visit roughly all 100,000 commits plus all their trees and blobs — potentially millions of objects total.

Q6.2 — GC race condition?

Suppose a commit operation has just written a new blob to the object store but hasn't yet written the tree or commit that references it. At that exact moment GC runs, scans all refs, finds the blob is unreachable (no commit points to it yet), and deletes it. Now when the commit operation tries to write the tree referencing that blob, the blob is gone and the commit is corrupt. Git avoids this by giving objects a grace period — any object newer than a certain age (default 2 weeks) is never deleted by GC, ensuring in-progress operations always finish before their objects become eligible for collection.